



## TREMPIST - טרמפיסט

מסמך עיצוב

שני גל

גרסה 1.0

3.12.25

### היסטוריית גרסאות המסמך

| תאריך     | גרסה | תקציר השינויים |
|-----------|------|----------------|
| 3.12.2025 | 1    | מסמך ראשוני    |
|           |      |                |
|           |      |                |
|           |      |                |

## מסמך עיצוב פרויקט



## 1. הקדמה

החלק הזה ישמש להצגה כללית של מסמך העיצוב. הסבר כללי, וקישור למסמכים קודמים כמו מסמך האפיון עליו אנו מתבססים

### 1.1 מטרה

המטרה של המסמך היא להסביר איך אפליקציית TREMPIST תהיה בנויה בפועל – איך ייראו החלקים השונים של המערכת ואיך הם יעבדו ביחד. המסמך עוזר להבין את הכיוון לפני שאתחיל לפתח.

המסמך מיועד ל:

- צביקה
- בשבילי, מנהלת הפרוייקט, לסדר את התכנית של הפיתוח
- חברי לכיתה
- הבוחן
- כל מי שייקח חלק בפרוייקט הזה

### 1.2 המוצר

#### **TREMPIST – טרמפיסט**

TREMPIST היא אפליקציה קהילתית לשיתוף טרמפים, שמבוססת על הדדיות ולא על כסף. כל משתמש יכול גם לתת וגם לקבל טרמפים, לפי מודל קרדיטים פשוט: נותנים טרמפ – מקבלים קרדיט, מקבלים טרמפ – משתמשים בקרדיט.

האפליקציה מאפשרת לפרסם טרמפים, לחפש טרמפים בזמן אמת, לראות מסלולים על מפה, לקבל התאמות אוטומטיות ולדרג משתמשים אחרי נסיעה.

### 1.3 קישור למסמכים קודמים

[שני גל - מסמך אפיון](#)

### 1.4

#### הגדרות:

**טרמפיסט** – משתמש שמציע או מבקש טרמפ.

**קרדיט** – ה"תשלום" עבור נסיעה, יחידת ערך שמייצגת "נתתי טרמפ ולכן מגיע לי לקבל".

**התאמה** – מנגנון שזיהה חפיפה במסלול ובזמן בין נהג לנוסע.

**Admin** – משתמש שמפקח על תקינות המערכת.

## 2. ארכיטקטורת המערכת

חלק זה כולל את תיאור מבנה המערכת ופירוט המודולים השונים בה

### 2.1 מבט על

מערכת TREMPIST בנויה ממספר מודולים מרכזיים, שכל אחד מהם אחראי על חלק אחר בתהליך של מציאת/פרסום טרמפ ושמירה על ההדדיות בקהילה. המודולים עובדים יחד כדי לאפשר חוויית שימוש רציפה, מהירה ובטוחה.

### 1. אפליקציית המובייל- לקוח (Client)

האפליקציה שבה המשתמשים משתמשים בפועל.  
אחראית על:

- הצגת מסכים (פרופיל, מציאת טרמפ, הצעת טרמפ, מפה)
- שליחת בקשות לשרת (כמו חיפוש טרמפ או פרסום טרמפ)
- קבלת התראות והתאמות
- הצגת הקרדיטים של המשתמש

היא משתמשת בנתונים שמגיעים מהשרת וב-GPS שמגיע מהמכשיר.

### 2. שרת (Server)

זהו החלק שמבצע את כל הלוגיקה "מאחורי הקלעים".  
תחומי אחריות:

- ניהול הקרדיטים (בדיקה, הוספה, הורדה)
- התאמה בין נהגים לנוסעים לפי נתיב, זמן ומרחק
- שמירת כל הנתונים בבסיס הנתונים
- אימות משתמשים
- ניהול דירוגים

- טיפול בדיווחים וחסימות

השרת הוא זה שמבצע את ההחלטות - לדוגמה האם למשתמש יש מספיק קרדיטים כדי לבקש טרמפ.

### 3. בסיס הנתונים (Database)

מקום שבו נשמר כל המידע:

- משתמשים
- טרמפים פתוחים וטרמפים שהסתיימו
- דירוגים
- קרדיטים
- היסטוריית נסיעות
- משתמשים בעייתיים (לאדמין)

הבסיס נתונים בנוי כך שאפשר לבצע חיפושים מהירים במיוחד - כי התאמת טרמפ חייבת לקרות בזמן אמת.

### 4. Google Maps API

רכיב שמספק:

- מיקום בזמן אמת
- חישוב מסלולים
- מרחקים בין נהגים לנוסעים
- תצוגת מפה באפליקציה

ללא Maps המערכת לא הייתה יכולה לבצע התאמות חכמות.

### אני אשתמש בחלוקה הזאת כי:

- **מודולריות** – כל חלק במערכת עושה דבר אחד בצורה טובה, מה שמקל על פיתוח, תיקונים והרחבות בעתיד.

## מסמך עיצוב פרויקט

- **ביצועים בזמן אמת** – התאמות טרמפים צריכות להיעשות מהר, ולכן השרת לוקח על עצמו את החישובים הכבדים.
- **גמישות עתידית** – ניתן להחליף רכיבים (למשל סוג בסיס נתונים או הוספת לוגיקת AI) בלי לשכתב את כל המערכת.
- **אבטחה** – הקרדיטים מנוהלים בצד השרת כדי למנוע מניפולציות של משתמשים.

## 2.2 פירוט רכיבי המערכת

המערכת מורכבת ממספר מודולים עיקריים. כל מודול אחראי על חלק מוגדר במערכת, ויחד הם מרכיבים את הזרימה הכללית של TREMPIST — מאימות משתמש, דרך התאמה בזמן אמת ועד ניהול הקרדיטים. כל מודול כולל מחלקות, פונקציות ואובייקטים שהאפליקציה משתמשת בהם, וכולם פועלים בצורה ביחד כדי לאפשר תחזוקה, הרחבה ושינוי בעתיד ללא שבירת המערכת.

## 1. מודול ניהול משתמשים (User Management)

**תפקיד עיקרי:** אחראי על הרשמה, התחברות, שמירת פרטים אישיים, שליפת פרופיל וניהול סטטוס משתמש (כולל חסימות ע"י אדמין).

### מחלקות מרכזיות

#### User

- מכיל פרטים בסיסיים: שם, טלפון, אימייל, תמונה, דירוג, כמות קרדיטים.
- מאפיינים:

- public: getters עבור נתוני המשתמש
- private: סיסמה, טוקן, סטטוס חסימה

#### UserAuth

תפקיד:

- התחברות

## מסמך עיצוב פרויקט

- אימות משתמש
- יצירת טוקן
- שמירה על Session

פונקציות:

- login(email, password) → מחזירה Token או שגיאה
- register(userData) → יוצר משתמש חדש
- verifyToken(token)
- 

## Data Flow

1. המשתמש מזין פרטים → נשלח לשרת
2. השרת מאמת מול DB
3. מחזיר טוקן וזיהוי משתמש
4. האפליקציה משתמשת בטוקן לכל פעולה מאובטחת

## 2. מודול הטרמפים (Rides Module)

תפקיד עיקרי: ניהול פרסום טרמפים, חיפוש טרמפים והצגת פרטי נסיעה.

### מחלקות מרכזיות

## Ride

מכילה מידע על טרמפ:

- נקודת יציאה
- יעד

## מסמך עיצוב פרויקט

- זמן יציאה
- מספר מקומות
- מזהה נהג
- סטטוס טרמפ

### RideSearch

אחראית על:

- חיפוש טרמפים לפי מיקום
- סינון לפי העדפות
- סידור התוצאות לפי רלוונטיות

פונקציות:

- $\text{search}(\text{from}, \text{to}, \text{time}) \rightarrow$  מחזירה רשימת טרמפים

### Data Flow

- נוסע מבקש חיפוש  $\rightarrow$  המערכת מבצעת שאילתה על בסיס הנתונים
- נשלפת רשימת טרמפים "פתוחים"
- מסוננת לפי מסלול, שעה וקרבה
- מוחזרת למשתמש

## 3. מנוע ההתאמה בזמן אמת (Matching Engine)

תפקיד עיקרי: לב המערכת — ההתאמה בין נהגים לנוסעים.

מחלקה מרכזית

### MatchingEngine



## מסמך עיצוב פרויקט

פונקציות:

- (compareRoutes(driverRoute, passengerRoute
- (calculateDistance(a, b
- (matchRide(ride, passenger

שימוש ב-Google Maps לחישוב מסלולים.

### Data Flow

1. נהג מפרסם טרמפ → נשמר ב-DB
2. נוסע מחפש → המנוע מחשב התאמות
3. אם נמצאה התאמה:
  - נשלחת הודעה לשני הצדדים
  - הסטטוס של הטרמפ מתעדכן

## 4. מודול הקרדיטים (Credits System)

תפקיד עיקרי: שמירה על ההדדיות: מי שמעניק טרמפ יכול לקבל טרמפ.

### מחלקה מרכזית

### CreditManager

פונקציות:

- (addCredit(userID
- (removeCredit(userID
- (canRequestRide(userID

### Data Flow

- בסיום טרמפ → CreditManager מוסיף +1 לנהג
- נוסע שמבקש טרמפ → בודק אם יש לפחות 1 קרדיט
- אם אין → המערכת לא מאפשרת בקשת טרמפ

## 5. מודול הדירוגים והביקורות (Ratings Module)

תפקיד עיקרי: בניית אמון בין משתמשים.

### מחלקות מרכזיות

#### Rating

מכילה:

- מזהה מדרג
- מזהה מדורג
- כוכבים 5–1
- תגובה טקסטואלית

#### RatingManager

- addRating(ratingData)
- calculateAverage(userID)

#### Data Flow

1. לאחר כל טרמפ - נפתחת בקשה לדירוג
2. המשתמש מדורג
3. השרת מחשב ממוצע
4. הממוצע מוצג בפרופיל

## 6. מודול המפות (Maps & GPS Module)

תפקיד עיקרי: מיקום בזמן אמת, חישוב מסלולים, תצוגת מפה.

### מחלקות

#### MapService

- שימוש ב-Google Maps API
- `getRoute(from, to)`
- `getLiveLocation()`

#### Data Flow

- האפליקציה מבקשת מיקום מהמכשיר
- שולחת לשרת (לא תמיד חובה)
- מוצג על המפה ומבטיח התאמה מדויקת

## 7. מודול התקשורת (Network Module)

מודול תשתיתי שאחראי על:

- שליחת בקשות לשרת (REST API)
- טיפול בשגיאות
- ניהול טוקנים
- `Retry` על פעולות שנכשלו

מחלקות:

- `ApiClient`
- `RequestHandler`

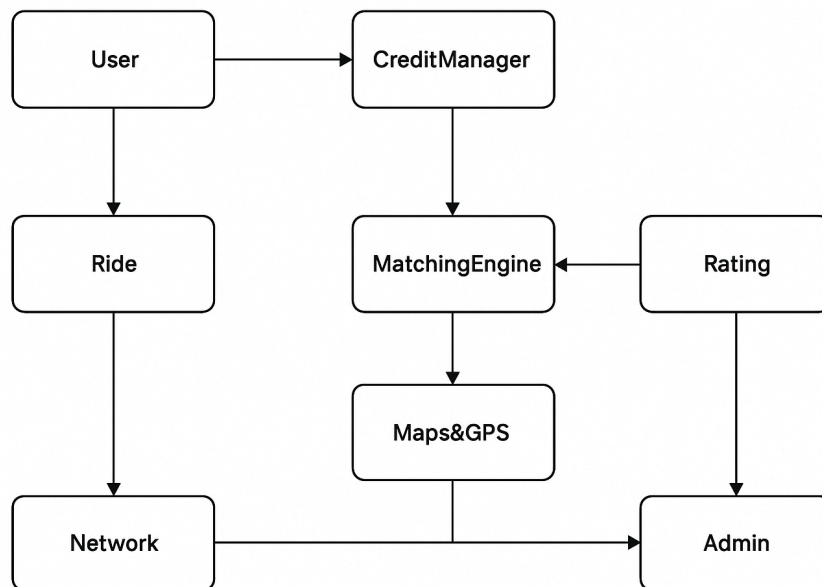
## 8. מודול אדמין (Admin Module)

תפקיד עיקרי: ניהול קהילה ושמירה על תקינות.

### פונקציות

- חסימת משתמש
- צפייה בדיווחים
- מחיקת טרמפים
- מעקב אחרי התנהגות חריגה

### הקשר בין המודולים:



## 2.3 דיון בנושא העיצוב הנבחר

המערכת חולקה למודולים נפרדים (משתמשים, טרמפים, התאמות, קרדיטים, דירוגים, מפות, אדמין ותקשורת) כדי שיהיה קל לפתח כל חלק בנפרד, להבין את הזרימה במערכת ולבצע שינויים בעתיד בלי לפגוע ברכיבים אחרים.

### למה בחרתי בחלוקה הזו?

- מבנה מודולרי מאפשר תחזוקה קלה והרחבה עתידית.
- מנוע ההתאמה דורש לוגיקה נפרדת בגלל חישובי מסלולים.
- מערכת הקרדיטים חייבת להיות מבודדת מטעמי אבטחה והוגנות.
- המפות מנוהלות כרכיב עצמאי בגלל תלות ב-Google Maps.

### יתרונות

- פיתוח ברור ומסודר.
- התאמות בזמן אמת יעילות ומהירות.
- קל להוסיף תכונות חדשות בהמשך.
- אבטחה טובה יותר על נתונים רגישים (קרדיטים ודירוגים).

### חסרונות

- תלות מלאה בשרת ובאינטרנט.
- מנוע ההתאמה דורש משאבי שרת.
- יותר מודולים משמעותם יותר תיאום ביניהם.

### חלופות שנשקלו

- מערכת "מונוליתית" אחת - נדחה בגלל קושי לתחזק.
- התאמות בלי מנוע חכם - נדחה כי פוגע בחוויית משתמש.

## מסמך עיצוב פרויקט

- ניהול קרדיטים באפליקציה → נדחה בגלל אבטחה.

## בחירת טכנולוגיות

- Flutter לאפליקציה – תומך ב־Android ועשוי לתמוך ב־iOS בהמשך.
- שרת Node.js / Python – מתאים ללוגיקה בזמן אמת.
- Google Maps API – מדויק ומתאים להתאמות מבוססות מיקום.

3. עיצוב נתונים ופרוטוקולים

User 3.1.1 — נתוני משתמש

| שדה           | סוג       | תיאור               | טווח ערכים / אילוצים | האם חובה      |
|---------------|-----------|---------------------|----------------------|---------------|
| user_id       | String    | מזהה משתמש ייחודי   | GUID                 | כן            |
| name          | String    | שם מלא              | 40-2 תווים           | כן            |
| email         | String    | אימייל ייחודי       | חייב פורמט תקין      | כן            |
| phone         | String    | מספר טלפון          | 10 ספרות             | כן            |
| password_hash | String    | סיסמה מוצפנת        | SHA-256              | כן            |
| rating        | Float     | ממוצע דירוגים       | 5.0-1.0              | ברירת מחדל: 5 |
| credits       | Int       | מאזן קרדיטים        | 0 ומעלה בלבד         | כן            |
| is_blocked    | Boolean   | האם המשתמש חסום     | true/false           | לא            |
| profile_pic   | String    | קישור לתמונת פרופיל | יכול להיות ריק       | לא            |
| created_at    | Timestamp | זמן יצירה           | אוטומטי              | כן            |

Ride 3.1.2 — נתוני טרמפ

| שדה             | סוג       | תיאור                            | טווח/אילוצים            | חובה |
|-----------------|-----------|----------------------------------|-------------------------|------|
| ride_id         | String    | מזהה טרמפ                        | GUID                    | כן   |
| driver_id       | String    | מזהה נהג                         | חייב להיות user_id קיים | כן   |
| origin          | String    | נקודת מוצא                       | לא ריק                  | כן   |
| destination     | String    | יעד                              | לא ריק                  | כן   |
| departure_time  | DateTime  | שעת יציאה                        | תאריך עתידי בלבד        | כן   |
| seats_available | Int       | מספר מקומות פנויים               | 6-1                     | כן   |
| status          | Enum      | OPEN / MATCHED / DONE / CANCELED | ערך תקין בלבד           | כן   |
| created_at      | Timestamp | זמן פרסום                        | אוטומטי                 | כן   |



**Match 3.1.3 — התאמה בין נהג לנוסע**

| שדה                 | סוג     | תיאור                         | אילוצים          |
|---------------------|---------|-------------------------------|------------------|
| match_id            | String  | מזהה התאמה                    | GUID             |
| ride_id             | String  | מזהה טרמפ                     | קיים בטבלת Rides |
| passenger_id        | String  | מזהה נוסע                     | קיים בטבלת Users |
| driver_confirmed    | Boolean | האם הנהג אישר                 | true/false       |
| passenger_confirmed | Boolean | האם הנוסע אישר                | true/false       |
| status              | Enum    | PENDING / APPROVED / REJECTED | חובה             |

**Rating 3.1.4 — דירוג**

| שדה        | סוג       | תיאור          | אילוצים        |
|------------|-----------|----------------|----------------|
| rating_id  | String    | מזהה דירוג     | GUID           |
| from_user  | String    | המדרג          | קיים ב־Users   |
| to_user    | String    | המדורג         | קיים ב־Users   |
| stars      | Int       | ציון           | 5–1            |
| comment    | String    | הערה טקסטואלית | יכול להיות ריק |
| created_at | Timestamp | זמן הדירוג     | אוטומטי        |

## PositionUpdate 3.1.5 — עדכון מיקום

שימוש במיקום חי של נהג/נוסע לצורך התאמה.

שדות:

- user\_id — מזהה משתמש
- lat — קו רוחב (float)
- lng — קו אורך (float)
- timestamp — זמן הדיווח

אילוצים:

lat: -90 עד 90  
lng: -180 עד 180

## CreditsLog 3.1.6 — היסטוריית קרדיטים

| שדה          | סוג       | תיאור                          |
|--------------|-----------|--------------------------------|
| log_id       | String    | מזהה רשומה                     |
| user_id      | String    | מזהה משתמש                     |
| change       | Int       | 1+ נהג / -1 נוסע               |
| reason       | Enum      | RIDE_COMPLETE / REQUEST_DENIED |
| related_ride | String    | מזהה טרמפ                      |
| timestamp    | Timestamp | זמן פעולה                      |

הערה לגבי השדה של charge בקרדיטים:  
מי שנתן טרמפ (כלומר שימש כנהג בנסיעה הזאת) → מקבל 1+ קרדיט  
מי שקיבל טרמפ (שימש כנוסע בנסיעה הזאת) → משלם 1- קרדיט  
אין באמת נהג/נוסע כי כולם הם גם נהגים וגם נוסעים.

## 3.2 פרוטוקול תקשורת:

התקשורת בין האפליקציה לשרת מבוססת על REST API בפורמט JSON ועל גבי HTTPS.

### 3.2.1 מצבי תקשורת (Communication States)

#### 1. הרשמה / Register

- קלט: name, email, phone, password
- פלט אפשרי:
  - הצלחה → OK
  - E002 → אימייל קיים
  - E010 → שדות חסרים
  - E011 → סיסמה חלשה

#### 2. התחברות / Login

- קלט: email, password
- פלט:
  - OK + token
  - E001 → סיסמה שגויה
  - E012 → משתמש חסום

#### 3. פרסום טרמפ

- קלט: נתוני טרמפ
- תגובות:
  - OK
  - E013 → פרטים חסרים

#### 4. חיפוש טרמפ

- קלט: origin, destination
- פלט:
- רשימת טרמפים
- E005 → לא נמצאו תוצאות

#### 5. בקשת התאמה

- פלט:
- OK
- E003 → אין מספיק קרדיטים
- E004 → אין מקומות פנויים

#### 6. אישור התאמה

- שני הצדדים צריכים לאשר → ואז נוצרת התאמה סופית.

#### 7. עדכוני מיקום

- נשלחים כל 5–15 שניות (לפי רענון המפה).

#### 8. דירוג נסיעה

- קלט: stars, comment
- פלט:
- OK
- E014 → אי אפשר לדרג משתמש שלא נסעת איתו

### 3.3 קודי API אחידים

טבלה של כל קודי הAPI שאשתמש בהם והסבר:

| קוד     | משמעות            |
|---------|-------------------|
| OK      | פעולה בוצעה       |
| ERROR   | תקלה כללית        |
| E001    | סיסמה שגויה       |
| E002    | אימייל כבר רשום   |
| E003    | אין מספיק קרדיטים |
| E004    | אין מקומות פנויים |
| E005    | לא נמצאו תוצאות   |
| E010    | שדות חסרים        |
| E011    | סיסמה חלשה        |
| E012    | משתמש חסום        |
| NO_AUTH | טוקן לא תקין      |

## 4. ממשק משתמש:

### תרשים זרימה – תהליך התחברות

#### 1. המשתמש מזין מייל + סיסמה

→ נשלח לשרת

#### 2. השרת בודק:

- האם האימייל קיים?
- האם הסיסמה תואמת?
- האם המשתמש חסום?

#### 3. תגובה:

- OK + token
- או
- שגיאה מתאימה (E001/E012 וכו')

### סקיצה:

